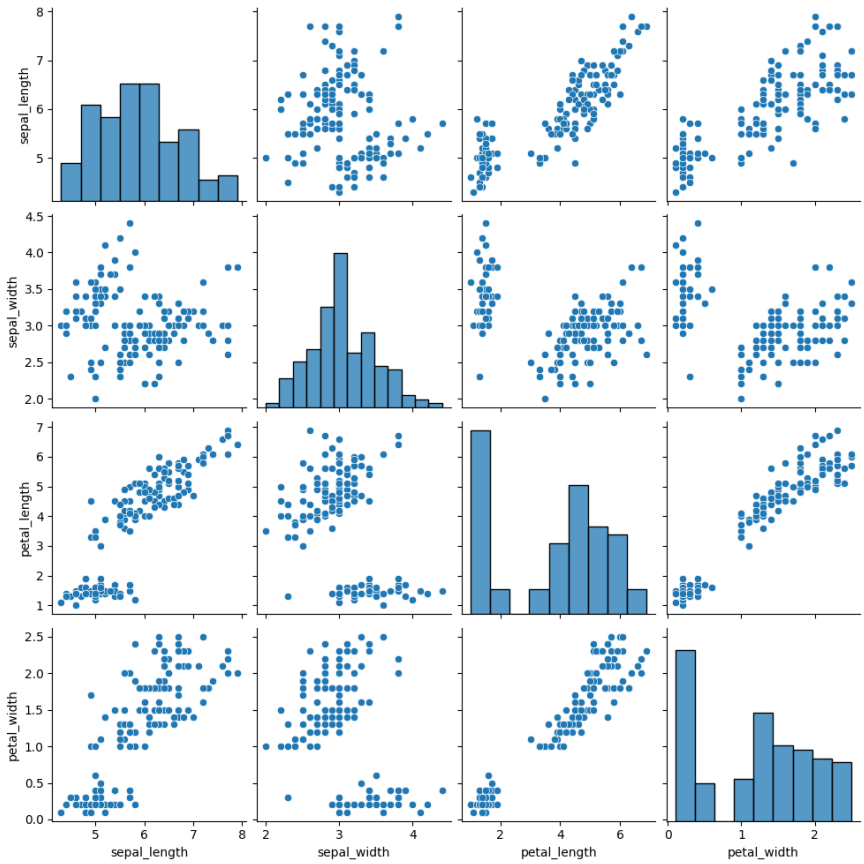
Correlation Regression and Analysis of Variance

Correlation refers to some statistical relationships involving dependence between two data sets.

While linear regression is a linear approach to establish the relationship between a dependent variable and one or more independent variables.

A single independent variable is called linear regression whereas multiple independent variables is called multiple regression.

```
import matplotlib.pyplot as plt
import seaborn as sns
df = sns.load_dataset('iris')
#without regression
sns.pairplot(df, kind="scatter")
plt.show()
```



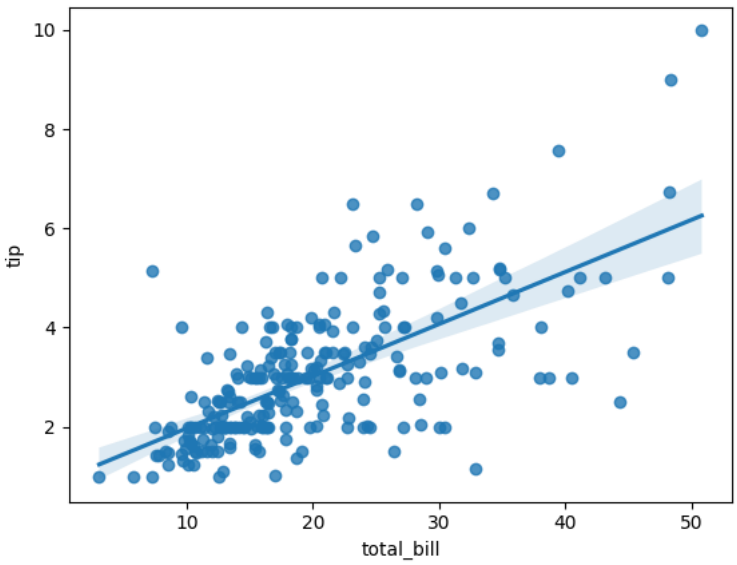
Linear Regression

Mathematically a linear relationship represents a straight line when plotted as a graph.

A non-linear relationship where the exponent of any variable is not equal to 1 creates a curve.

The functions in Seaborn to find the linear regression relationship is regplot.

```
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('tips')
sb.regplot(x = "total_bill", y = "tip", data = df)
plt.show()
```



Example

Import the necessary packages such as numpy, matplotlib and scipy

```
import matplotlib.pyplot as plt
from scipy import stats
```

Create a an array for X and Y axis

```
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

Execute a method that returns some important key values of Linear Regression:

```
slope, intercept, r, p, std_err = stats.linregress(x, y)
slope
#intercept
```

```
-1.7512877115526118
```

Create a function that uses the slope and intercept values to return a new value. This new value represents where on the y-axis the corresponding x value will be placed:

```
def myfunc(x):
  return slope * x + intercept
```

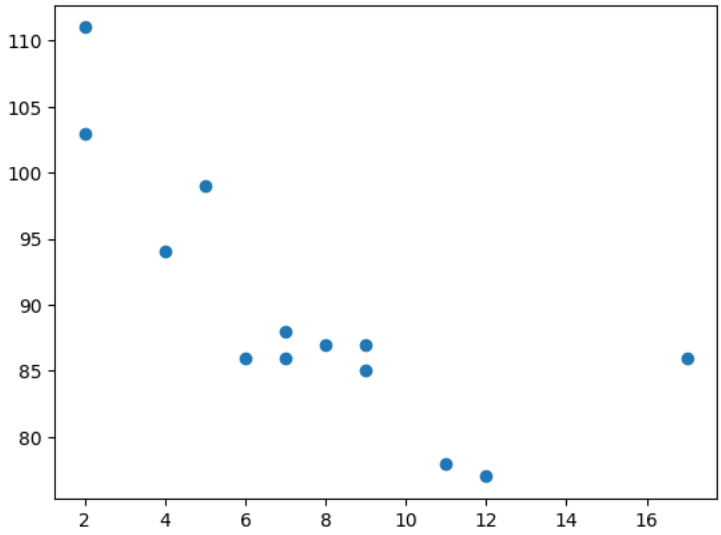
Run each value of the x array through the function. This will result in a new array with new values for the y-axis:

```
mymodel = list(map(myfunc, x))
```

Draw the original scatter plot:

```
plt.scatter(x, y)
```

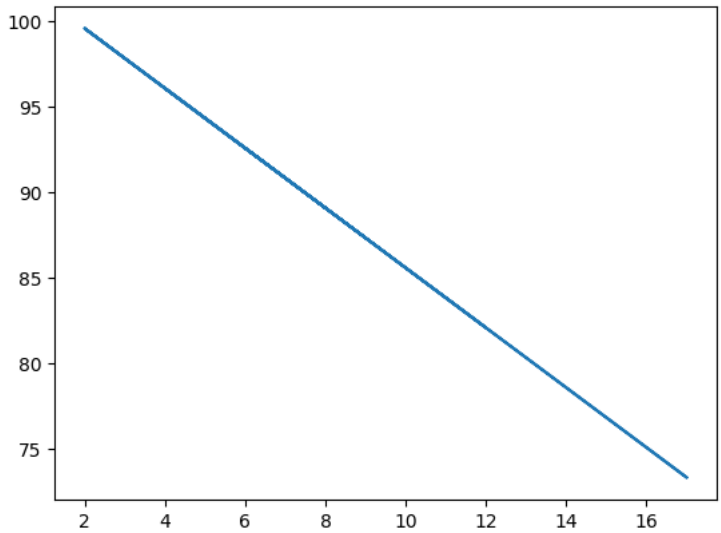
```
<matplotlib.collections.PathCollection at 0x7992682afe50>
```



Draw the line of linear regression:

```
plt.plot(x, mymodel)
```

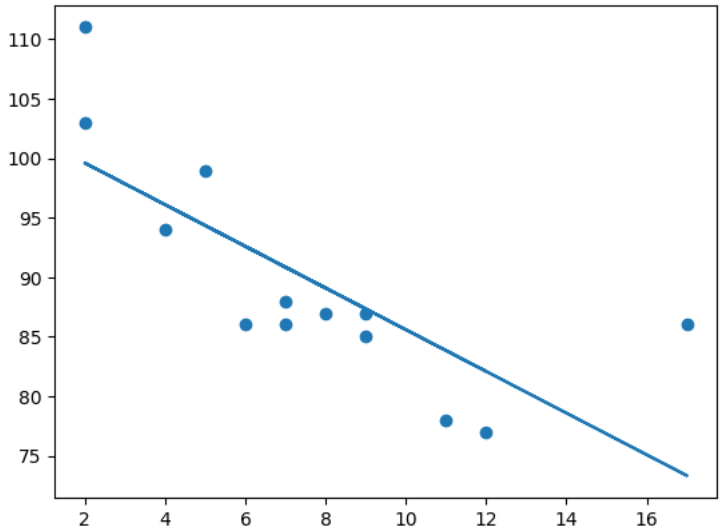
```
[<matplotlib.lines.Line2D at 0x799267674070>]
```



Display the diagram:

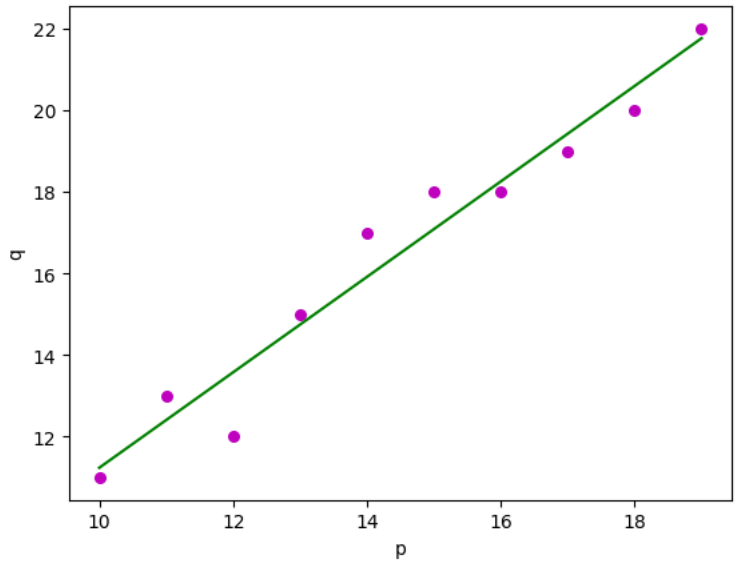
```
plt.show()
```

```
import matplotlib.pyplot as plt
from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
  return slope * x + intercept

mymodel = list(map(myfunc, x))

plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```




```
import numpy as nmp
import matplotlib.pyplot as mtplt

def estimate_coeff(p, q):
# Here, we will estimate the total number of points or observation
    n1 = nmp.size(p)
# Now, we will calculate the mean of a and b vector
    m_p = nmp.mean(p)
    m_q = nmp.mean(q)

# here, we will calculate the cross deviation and deviation about a
    SS_pq = nmp.sum(q * p) - n1 * m_q * m_p
    SS_pp = nmp.sum(p * p) - n1 * m_p * m_p

# here, we will calculate the regression coefficients
    b_1 = SS_pq / SS_pp
    b_0 = m_q - b_1 * m_p

    return (b_0, b_1)

def plot_regression_line(p, q, b):
# Now, we will plot the actual points or observation as scatter plot
    mtplt.scatter(p, q, color = "m",
            marker = "o", s = 30)

# here, we will calculate the predicted response vector
    q_pred = b[0] + b[1] * p

# here, we will plot the regression line
    mtplt.plot(p, q_pred, color = "g")

# here, we will put the labels
    mtplt.xlabel('p')
    mtplt.ylabel('q')

# here, we will define the function to show plot
    mtplt.show()

def main():
# entering the observation points or data
    p = nmp.array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
    q = nmp.array([11, 13, 12, 15, 17, 18, 18, 19, 20, 22])

# now, we will estimate the coefficients
    b = estimate_coeff(p, q)
    print("Estimated coefficients are :\nb_0 = {} \ \nb_1 = {}".format(b[0], b[1]))

# Now, we will plot the regression line
    plot_regression_line(p, q, b)

if __name__ == "__main__":
    main()
```

```
Estimated coefficients are :
b_0 = -0.4606060606060609 \
b_1 = 1.1696969696969697
```



Start coding or generate with AI.